

SmartStudyAgent 反思报告

1. 项目概述

SmartStudyAgent 是一个基于 .NET 8 的智能学习助手 Agent 系统。它的目标不是做一个单纯聊天机器人，而是围绕用户上传的学习资料完成检索、总结、出题、学习计划制定、知识点整理等任务。

项目采用 ASP.NET Core Minimal API 作为后端入口，前端页面位于 `wwwroot`，后端核心由 `StudyAgent`、多个子 Agent、工具注册表、资料服务、记忆服务、RAG 服务和 LLM 服务组成。

系统核心能力包括：

- 通过 OpenAI-compatible Chat Completions API 调用大模型。
- 使用 `StudyAgent.RunAsync` 实现 Agent Loop。
- 注册 6 个自定义工具，供 Agent 根据任务调用。
- 使用短期记忆和长期记忆保存上下文。
- 支持资料上传、文本提取、OCR、RAG 检索和 Web 页面交互。
- 通过多个子 Agent 分工处理资料、练习题、计划和知识点整理任务。

2. Agent 内部工作原理

本项目中的 Agent 由 4 个关键部分组成：

组件	项目中的实现	作用
LLM	<code>ILlmService</code> 、 <code>OpenAiCompatibleLlmService</code>	负责根据系统提示词、用户请求、工具结果和记忆生成自然语言回答
Agent Loop	<code>StudyAgent.RunAsync</code>	控制“思考、行动、观察、回答”的执行流程
Tools	<code>IStudyTool</code> 、 <code>StudyToolRegistry</code> 和 6 个工具类	执行资料检索、总结、出题、计划等具体任务
Memory	<code>ConversationMemory</code> 、 <code>LongTermMemoryStore</code>	保存短期对话上下文和长期学习偏好

一次用户提问进入系统后，流程如下：

1. 前端调用 `/api/agent/chat` 或 `/api/agent/chat/stream`。
2. API 将请求交给 `StudyAgent.RunAsync`。
3. `StudyAgent` 保存用户消息到短期记忆。
4. `StudyAgent` 根据用户问题生成工具调用计划 `ToolCallPlan`。
5. 如果用户在前端选择了资料，系统会把资料 ID 注入工具参数。
6. `StudyAgent` 根据工具名选择合适的子 Agent。
7. 子 Agent 调用 `StudyToolRegistry`，找到并执行具体工具。
8. 工具返回 `ToolExecutionResult`，其中的 `Observation` 表示工具观察结果。
9. `StudyAgent` 把 Thought、Action、Observation 保存为 `AgentStep`。
10. 系统结合工具结果、短期记忆、长期记忆和提示词，调用 LLM 生成最终回答。

11. 回答写回短期记忆，并返回给前端显示。

这个过程体现了 Agent 和普通聊天机器人的区别：普通聊天机器人通常直接把用户问题交给模型回答，而本项目的 Agent 会先分析任务，调用工具获取外部信息，再基于工具结果生成回答。

3. ReAct 模式理解

ReAct 是 Reasoning and Acting 的缩写，核心思想是让 Agent 在推理和行动之间循环：

- Thought：分析当前用户意图，决定下一步做什么。
- Action：调用工具执行具体任务。
- Observation：读取工具返回结果。
- Final Answer：基于观察结果生成最终回答。

本项目实现的是轻量级 ReAct 流程。它没有让 LLM 自由输出 JSON 工具调用，而是使用代码中的规则规划器 `PlanNextAction` 做工具选择。这样设计的原因是：

- 对课程资料问答这类场景，用户意图相对明确，关键词规划可以稳定覆盖主要任务。
- 代码逻辑更容易解释和调试，适合小型项目和课堂演示。
- 避免模型输出不稳定 JSON 导致工具解析失败。
- 保留了后续升级空间，可以把 `PlanNextAction` 替换成真正的 LLM Function Calling。

虽然工具选择由规则完成，但整个系统仍然具备 Agent 的关键特征：它有任务规划、有工具调用、有观察结果、有记忆，并且最终回答会综合工具结果和上下文生成。

4. 核心循环代码解读

核心循环位于 `Agents/StudyAgent.cs` 的 `RunAsync` 方法。

4.1 会话 ID 处理

```
var sessionId = string.IsNullOrEmpty(request.SessionId)
    ? Guid.NewGuid().ToString("N")
    : request.SessionId;
```

这段代码决定当前请求属于哪个会话。如果前端没有传入 `SessionId`，系统就生成一个新的 GUID；如果传入了，就复用原来的会话。这样可以支持多轮对话和会话切换。

4.2 写入短期记忆

```
_memory.Add(sessionId, "user", request.Message);
```

这行代码把用户本次问题写入 `ConversationMemory`。后续生成最终回答时，系统会读取最近几条消息，帮助模型理解上下文。

4.3 初始化执行轨迹

```
var steps = new List<AgentStep>();
var maxSteps = Math.Clamp(request.MaxSteps, 1, 8);
var observations = new List<string>();
```

`steps` 用来保存前端展示的 Agent 执行轨迹。`maxSteps` 限制最大执行轮数，避免 Agent 无限循环。`observations` 保存工具返回的观察结果，帮助判断是否已经可以生成最终回答。

4.4 Agent Loop

```
for (var step = 1; step <= maxSteps; step++)
{
    var plan = PlanNextAction(request.Message, observations);
    AddSelectedMaterials(plan, request.MaterialIds);
    if (plan.ToolName.Equals("finish", StringComparison.OrdinalIgnoreCase))
    {
        break;
    }

    var subAgent = SelectSubAgent(plan.ToolName);
    ...
}
```

循环每一轮都会先调用 `PlanNextAction` 生成计划。计划里包含：

- `Thought`：为什么选择这个工具。
- `ToolName`：要调用哪个工具。
- `Arguments`：工具需要的参数。

随后 `AddSelectedMaterials` 会把前端选中的资料 ID 写入工具参数，确保 Agent 只围绕指定资料回答。如果计划是 `finish`，说明已有工具结果，可以退出工具调用阶段。

4.5 子 Agent 分派

```
var subAgent = SelectSubAgent(plan.ToolName);
```

`SelectSubAgent` 会遍历所有实现 `IStudySubAgent` 的子 Agent，找到能处理当前工具名的 Agent。例如：

- `search_materials` 交给 `MaterialAgent`。
- `generate_quiz` 交给 `PracticeAgent`。
- `create_study_plan` 交给 `PlanningAgent`。
- `extract_learning_points` 交给 `InsightAgent`。

这样做的好处是让不同类型任务有清晰边界，后续扩展新能力时可以新增子 Agent。

4.6 执行工具并记录观察结果

```
var result = await subAgent.ExecuteAsync(plan, cancellationToken);
observations.Add(result.Observation);

steps.Add(new AgentStep(
    step,
    $"CoordinatorAgent -> {subAgent.Name}: {plan.Thought}",
    result.ToolName,
    result.Observation,
    subAgent.Name));
```

子 Agent 会通过 `StudyToolRegistry` 找到具体工具并执行。工具返回的 `Observation` 既会进入 `observations`，也会被包装成 `AgentStep` 返回给前端。前端看到的 ReAct Trace 就来自这里。

4.7 停止工具调用

```
if (ShouldStopAfterTool(plan.ToolName))
{
    break;
}
```

当前项目中的工具大多是一次调用即可支撑回答的任务，因此执行一次工具后就停止循环。这样可以避免重复调用工具造成响应慢或结果混乱。

4.8 生成最终回答

```
var answer = await BuildFinalAnswerAsync(
    sessionId,
    request.Message,
    steps,
    cancellationToken);
```

`BuildFinalAnswerAsync` 会根据工具执行轨迹、短期记忆、长期记忆和提示词生成最终回答。部分工具如总结、检索、出题已经能返回完整结果时，方法会直接返回工具结果，减少不必要的二次生成。

4.9 添加资料来源

```
answer = await AddMaterialSourceHeaderAsync(answer, request.MaterialIds,
    cancellationToken);
```

如果用户指定了资料，系统会在回答前添加“本次回答基于以下资料”。这样做可以让答案来源更清楚，减少用户误以为回答来自所有资料。

4.10 写入助手记忆并返回响应

```
_memory.Add(sessionId, "assistant", answer);

return new AgentChatResponse(
    sessionId,
    answer,
    steps,
    _memory.GetMessages(sessionId));
```

最终答案会写入短期记忆，方便后续追问。返回结果中包含答案、执行步骤和当前记忆，因此前端可以同时显示回答内容和 Agent 执行过程。

5. 工具调用机制

项目没有使用 Semantic Kernel Plugin，而是使用原生 C# 接口实现工具注册和调用。

核心接口是 `IStudyTool`：

```
public interface IStudyTool
{
    string Name { get; }
    string Description { get; }
    Task<ToolExecutionResult> ExecuteAsync(
        IReadOnlyDictionary<string, string> arguments,
        CancellationToken cancellationToken);
}
```

每个工具都有稳定的 `Name`，Agent 根据这个名称调用工具。`Description` 描述工具能力，`ExecuteAsync` 执行具体任务并返回观察结果。

`StudyToolRegistry` 统一保存所有工具：

- 构造函数中注入 6 个工具。
- 使用字典按工具名保存。
- `ExecuteAsync` 根据 `ToolCallPlan.ToolName` 找到对应工具。
- 找不到工具时返回错误观察结果，而不是让程序崩溃。

这种设计的优点是结构简单、类型明确、容易扩展。新增工具时只需要实现 `IStudyTool`，再在 `StudyToolRegistry` 和 `Program.cs` 中注册。

6. 记忆机制设计

项目中有两类记忆：

6.1 短期记忆

短期记忆由 `ConversationMemory` 实现，用于保存多轮对话历史。

主要特点：

- 按 `sessionId` 保存消息。
- 每条消息包含角色、内容和时间。
- 支持读取最近消息构建上下文。
- 使用本地 JSON 文件持久化，程序重启后仍能恢复。
- 使用 `lock` 保证多请求同时写入时不会破坏字典和文件。

短期记忆适合回答“刚才那个问题”“继续解释”等追问。

6.2 长期记忆

长期记忆由 `LongTermMemoryStore` 实现，用于保存用户学习目标和偏好。

它和普通聊天记录分开保存，避免长期偏好被大量聊天消息淹没。最终回答时，`BuildFinalAnswerAsync` 会把长期记忆写入 user prompt，例如学习目标和学习偏好。

7. RAG 设计决策

本项目实现了轻量本地 RAG，而不是直接接入向量数据库。

选择本地 RAG 的原因：

- 项目规模较小，资料量可控。
- 不需要额外部署 Qdrant、Milvus、pgvector 等服务。
- 本地运行成本低，便于调试和演示。
- 代码更容易解释，能清楚展示 RAG 的核心思想。

RAG 流程如下：

1. `DocumentService` 读取所有资料正文。
2. `VectorRagService` 把每份资料切成小片段。
3. `LocalEmbeddingService` 把问题和片段转换为 128 维本地哈希向量。
4. 系统计算问题向量和片段向量的余弦相似度。
5. 按相关性排序，返回最相关的片段。

这个方案不是严格意义上的深度语义 Embedding，但能在不依赖外部服务的情况下实现“根据资料找依据”的基本能力。

8. LLM 集成设计

LLM 调用由 `OpenAiCompatibleLlmService` 负责，并通过 `ILlmService` 抽象。

设计决策：

- 使用 OpenAI-compatible 接口，便于切换不同模型服务。
- 使用 `HttpClient` 直接发送请求，不绑定特定 SDK。
- 配置从 `IConfiguration` 读取，避免把模型参数写死。
- API 调用失败时回退到 Mock 响应，保证系统流程仍可演示。
- 系统提示词要求模型基于工具结果回答，减少幻觉。

Mock 模式的意义是：即使没有 API Key，仍然可以测试上传、检索、工具调用、记忆和前端展示等流程。这对开发和调试很有帮助。

9. 多 Agent 协作设计

项目中有一个协调 Agent 和多个子 Agent：

- **StudyAgent**：协调者，负责规划和分派任务。
- **MaterialAgent**：处理资料列表、检索和总结。
- **PracticeAgent**：处理练习题生成。
- **PlanningAgent**：处理学习计划。
- **InsightAgent**：处理知识点、关键词和复习提纲。

这样设计的原因是把不同能力拆开，避免一个类承担所有任务。虽然当前子 Agent 主要负责分派工具，但结构上已经具备扩展空间。例如以后可以让 **PracticeAgent** 增加题目质量检查，让 **PlanningAgent** 增加计划调整逻辑。

10. 关键设计决策反思

10.1 为什么使用规则规划器

使用规则规划器而不是完全由 LLM 决定工具调用，主要是为了稳定性和可解释性。用户说“总结”“出题”“计划”“资料列表”等关键词时，系统可以确定地选择对应工具。

不足是规则覆盖范围有限，如果用户表达方式很复杂，可能选错工具。后续可以升级为 LLM Function Calling，让模型输出结构化工具调用。

10.2 为什么工具返回文本 Observation

工具返回文本而不是复杂对象，是为了让观察结果可以直接进入 prompt，也方便前端展示 ReAct Trace。缺点是结构化程度不够，如果后续要做复杂推理，可以让工具返回结构化 JSON，再由 Agent 统一格式化。

10.3 为什么使用本地 JSON 存储

本地 JSON 文件简单直观，不需要数据库，适合单机运行。资料、记忆都能直接落盘。缺点是并发能力和查询能力有限，如果用户量变大，应迁移到数据库。

10.4 为什么要保留 Mock 模式

Mock 模式保证没有 API Key 时系统也能运行，尤其适合演示基础流程。它也能帮助区分“Agent 流程问题”和“外部模型服务问题”。

10.5 为什么加入资料来源提示

当用户选择了某些资料时，回答前显示资料来源，可以避免答案混入其他文档内容，也让用户知道回答依据是什么。这对学习场景很重要。

11. AI 工具使用情况

本项目开发过程中使用了 AI 工具辅助，但不是直接复制后完全不理解。AI 主要参与以下方面：

- 辅助设计项目结构，例如 Agent、Tools、Services、Memory 的分层。
- 辅助生成部分样板代码，例如接口、模型、服务注册和 API 路由。
- 辅助文档图片的生成。

- 辅助完善 README、架构说明和反思报告。

我自己需要重点理解和掌握的部分包括：

- `StudyAgent.RunAsync` 的完整执行流程。
- `PlanNextAction` 如何根据用户意图选择工具。
- `IStudyTool` 和 `StudyToolRegistry` 如何实现工具调用。
- `ConversationMemory` 和 `LongTermMemoryStore` 如何保存上下文。
- `VectorRagService` 和 `LocalEmbeddingService` 如何实现轻量 RAG。
- `OpenAiCompatibleLlmService` 如何调用真实模型并在失败时回退 Mock。
- 文件上传后如何经过文本提取、OCR、保存和检索。

对于 AI 生成或辅助修改的代码，我通过阅读、运行、构建和调试来确认其作用。比如 README 修改后运行 `dotnet build` 检查项目仍可编译；提示词修改后确认只改变提示词文本，不改变 Agent 逻辑。

12. 当前不足与改进方向

项目目前已经实现主要 Agent 能力，但仍有一些不足：

- 工具选择依赖关键词规则，泛化能力不如真正的 LLM Function Calling。
- 本地哈希向量只能近似表达语义，检索效果不如专业 Embedding 模型。
- 本地 JSON 存储适合单机，不适合高并发或多用户部署。
- 单元测试还不够完整，目前主要依赖冒烟测试。
- OCR 对复杂扫描件、表格和图片混排资料的效果有限。
- Mock 响应中仍有少量英文触发词，后续可以继续统一成中文。

后续可以改进为：

- 引入真正的工具调用 JSON Schema 或 Semantic Kernel Plugin。
- 接入专业 Embedding 模型和向量数据库。
- 增加单元测试、集成测试和端到端测试。
- 增加用户认证和多用户资料隔离。
- 加强文件上传安全检查和日志脱敏。

13. 总结

通过这个项目，我理解了 Agent 和普通聊天机器人的核心区别：Agent 不只是生成文本，而是会围绕目标进行任务规划，调用工具获取外部信息，观察工具结果，并结合记忆生成最终回答。

SmartStudyAgent 的实现重点在于把 Agent 的核心组件拆清楚：LLM 负责生成语言，Agent Loop 负责任务编排，Tools 负责执行能力，Memory 负责保存上下文，RAG 负责从资料中找依据。这样的结构让项目更容易解释、测试和扩展。